

API

1. What is REST API?

REST API (Representational State Transfer) is a web service architecture where resources are accessed and manipulated over HTTP using standard methods like GET, POST, PUT, DELETE, and responses are typically in JSON or XML.

2. What is the difference between SOAP and REST?

SOAP is protocol-based (XML only), strict standards. REST is architecture-based (JSON/XML), flexible, stateless.

SOAP - sending a registered letter (formal, tracked, heavy).

REST - sending a text message (quick, simple, flexible).

3. What is the structure of HTTP request and response?

Request – Method (GET/POST/etc), URL, Headers, and (optionally) Body.

Response – Status code, Headers, and Body (often JSON or XML).

4. Common HTTP Status Codes?

100 informational, 200 success, 300 redirection, 400 client error, 500 server error.

200 OK (Success)

201 Created (Resource added)

400 Bad Request (Client error)

401 Unauthorized (Missing auth)

404 Not Found (Resource missing)

500 Internal Server Error (Server failure)

5. How do you automate API testing in Java?

I use the RestAssured library combined with JUnit for validations. I structure requests, validate responses, status codes, and headers.

6. How to authenticate APIs?

Depending on the API, I use Basic Auth, Bearer Tokens (OAuth 2.0), API keys, or custom headers. In RestAssured, it can be set easily using `.auth()` methods.

7. How do you validate an API response?

I check the status code, response time, content type, headers, and body fields using assertions

8. What is JSON? How do you validate it in RestAssured?

JSON (JavaScript Object Notation) is a lightweight data format with key-value pairs for API requests and responses. In RestAssured, I validate JSON using *body()* method with matchers like *equalTo()* or by parsing and asserting values.

9. What are Query Parameters in API?

Query parameters are key-value pairs added to a URL after *?* to filter or modify a request, e.g., */users?role=admin*.

10. What is Serialization and Deserialization?

Serialization is converting a Java object into JSON or XML.

Deserialization is converting JSON/XML into a Java object.

Jackson library is used.

11. What are some examples of API testing in your project?

I test user authentication, courses list endpoints, payment APIs, and verify database updates after hitting specific endpoints. I also validate negative scenarios and security aspects.

12. How to send a POST request in RestAssured?

```
given().body(payload).when().post("/endpoint");
```

13. How to handle nested JSON?

I parse the JSON using *jsonPath()* method and navigate to nested elements.

```
response.jsonPath().getString("data[0].attributes.name");
```

14. What is the difference between Query and Path parameter?

Query parameters are optional filters (*?type=admin*), while path parameters are part of the endpoint structure itself (*/users/{id}*).

15. How to log requests and responses?

By using `.log().all()` for requests and `.then().log().all()` for responses to get full visibility during debugging.

16. How to extract value from json response?

`String value = response.jsonPath().getString("data.id");`

17. What is the difference between PUT and PATCH?

PUT fully replaces a resource. PATCH partially updates specific fields.

18. How to test file upload?

`given().multiPart("file", new
File("path/to/file")).when().post("/upload").then().statusCode(200);`

19. Can you please explain your API testing experience?

We automate end-to-end API testing by sending different types of requests (GET, POST, PUT, DELETE) using RestAssured.

We validate responses by checking status codes, headers, and body fields.

We also handle authentication tokens dynamically and use POJOs for serialization/deserialization to ensure clean code structure.

20. How do you use Postman for testing?

I use Postman for quick manual API validation and environment management. Once the endpoints are understood, I automate tests using Rest Assured with Jackson for scalable, maintainable testing.

21. What is the difference between authentication and authorization?

Both secure access, but serve different purposes:

- Authentication verifies who the user is. It confirms identity using credentials like username/password, biometrics, or API tokens.
Example: I test authentication by sending a valid login request and expecting 200 OK, and an invalid one expecting 401 Unauthorized.
- Authorization controls what the authenticated user can access or do, based on roles, permissions, or policies.
Example: I test authorization by sending a request with an "admin"

token expecting 200 OK, and a regular "user" token expecting 403 Forbidden for restricted endpoints.

22. What is CRUD?

CRUD stands for Create, Read, Update, Delete. Four basic operations that define how software applications interact with data. Create - post, read - get, update - put, patch, delete - delete.

23. Can you please explain the difference between HTTP and HTTPS?

Imagine HTTP is like sending a postcard—anyone handling it can read your message. HTTPS is like sending a locked briefcase—only the recipient has the key.

Technically, HTTPS encrypts data using SSL/TLS certificates, while HTTP sends everything as plain text. As testers, we always check for 'https://' and the padlock icon, especially on login pages.

24. Could you please describe how a client and server interact?

Think of it like ordering food:

Client (you): Asks for a burger by clicking a 'Submit Order' button.

Server (kitchen): Receives the request, makes the burger, and sends it back.

In testing, I verify both sides—like checking if the button works (client UI) and if the order appears in the database (server API).

25. How would you verify a site uses HTTPS for forms?

- Manual Check: Submit a form and look for 'https://' in the URL bar and the padlock icon.
- Automated Test: In Selenium, I'd assert the current URL starts with 'https': `assertTrue(driver.getCurrentUrl().startsWith("https://"));`

26. What's the first thing you check in API testing?

I start with the status codes—like a health report:

200 OK means 'good job, API!'

401 Unauthorized means 'you forgot credentials.'

For example, if our login API returns 200, I then validate the response body contains the user's token.

27. How do you test a new API endpoint?"

I follow the 'happy path, sad path, bad path' approach:

Happy Path: Valid request → 200 OK + correct response.

Sad Path: Invalid input (e.g., wrong email format) → 400 Bad Request.

Bad Path: Malicious input (e.g., SQL injection) → 403 Forbidden.

28. What is an Object Mapper? Why do you use it?

Object Mapper is like a translator between JSON/XML (API responses) and Java objects. For example, when an API returns this JSON:

```
{"name":"John","age":30}
```

An Object Mapper (like Jackson) converts it to a Java Person class automatically:

```
Person person = objectMapper.readValue(json, Person.class);
```

```
// Now person.getName() → "John"
```

We use it to avoid manual parsing—saving time and reducing errors.

29. Which Object Mappers have you used, and how do they differ?

Two main ones:

Jackson (Spring Boot's default):

Faster, more features (annotations, streaming).

Example: `@JsonProperty` to map odd field names.

Gson (Google's):

Simpler API, easier debugging.

Better for quick prototyping.

I prefer Jackson for enterprise projects due to its Spring integration.

30. How would you map a JSON field named 'user_name' to a Java field 'username'?

With Jackson's `@JsonProperty`:

```
public class User {  
    @JsonProperty("user_name") // ← Maps JSON → Java  
    private String username;  
}
```

Without this, the mapper expects exact field-name matches.

31. How do you configure an Object Mapper to ignore null fields?

Two ways:

Globally:

```
objectMapper.setSerializationInclusion(Include.NON_NULL);
```

Per-class:

```
@JsonInclude(Include.NON_NULL)
```

```
public class Product { ... }
```

This keeps JSON responses clean when sending data to APIs.

32. How would you format a Date field to 'yyyy-MM-dd' in JSON?

With @JsonFormat:

```
public class Event {  
    @JsonFormat(pattern="yyyy-MM-dd")  
    private Date eventDate;  
}
```

Or globally:

```
objectMapper.setDateFormat(new SimpleDateFormat("yyyy-MM-dd"));
```

This ensures consistency across all API responses.

33. What if JSON doesn't match your Java object's structure?

Jackson throws JsonProcessingException. I handle it like this:

```
try {  
    User user = objectMapper.readValue(json, User.class);  
} catch (JsonProcessingException e) {  
    log.error("Malformed JSON for User: {}", json);  
    throw new ApiTestException("Invalid JSON payload");  
}
```

Pro Tip: Use @JsonIgnoreProperties(ignoreUnknown=true) to skip unexpected fields.

34. How do you optimize Object Mapper usage in tests?

Two tricks:

Reuse the ObjectMapper (expensive to create):

```
public static final ObjectMapper MAPPER = new ObjectMapper();
```

Preconfigure it:

```
MAPPER.configure(DeserializationFeature.FAIL_ON_UNKNOWN_PROPERTIES, false);
```

This avoids recreating mappers for every test, speeding up execution.